

Symmetry Breaking in Dependency Quantified Boolean Formulas

Paul Seip, Clemens Hofstadler, Manuel Kauers, Martina Seidl
(Johannes Kepler University Linz)

August 17, 2026

Abstract

Symmetry breaking techniques have been extensively studied in automated reasoning for reducing the search space. In particular these techniques have already been well explored in the case of SAT and QBF. In this talk we discuss symmetries of dependency quantified boolean formulas (DQBF), a generalization of QBF which allows for explicit quantifier dependencies. It turns out that extending existing QBF symmetry techniques to DQBF is non-trivial due to a possible non-linear dependency structure. We illustrate these difficulties with an example that shows that a naive lifting of QBF symmetry breaking is not sufficient, and that we need more refined methods for dealing with such dependency structures.

1 Introduction

Symmetries play an important role in almost every area of science; they are used as a guiding principle to detect order in the object of study. Often – particularly in physics – scientists are not only interested in observing symmetry, but also in *breaking symmetry*.

Another area where symmetry breaking is prominent is *automated reasoning*. If one is able to detect symmetries in the search space one is dealing with, then collapsing these symmetries can significantly speed up the solving time, since the solver only has to focus on non-symmetric potential solutions. These symmetry breaking techniques have already been extensively used in SAT [1, 10, 11, 12, 20], CSP [8, 9, 14], and QBF [2, 3, 4] to improve solver performance. On the theoretical side, symmetries can enhance proof systems by introducing new deduction rules that exploit symmetries, ultimately resulting in exponentially more powerful proof systems [16, 18, 23]. Recent research has also focused on developing proof logging methods that can efficiently handle symmetry-reasoning [5, 22].

2 Dependency Quantified Boolean Formulas

Dependency Quantified Boolean formulas (DQBF) [6, 19] represent a rich and expressive class of logical formulas that extends QBF by allowing existentially quantified variables to depend on specific subsets of universally quantified variables. Every QBF induces an equivalent DQBF with a linearly ordered sequence of dependency sets. In contrast to QBFs, the nuanced quantification of DQBFs allows also for non-linear dependencies. This makes DQBF a powerful framework for encoding a variety of problems in verification, synthesis, and soft-/hardware engineering, see [7, 21] and references therein. Moreover, recent work has shown applications of DQBF model counting in Markov Logic Networks [13]. The extended expressive power, however, comes at the cost of increased computational complexity – the decision problem for

DQBF is NEXPTIME-complete [19]. This necessitates advanced methods for solving DQBF instances efficiently. One promising avenue for mitigating the inherent complexity in practice is the exploitation of symmetries.

3 Symmetry Breaking

The formal study of symmetry is rooted in group theory, which originated in the 19th century with Galois' insight that symmetry groups of polynomial roots provided the solution to a long-standing problem. In [17] two of the authors give a formal treatment of symmetries in QBF, using the language of group theory. Their main theorem [17, Theorem 22] constructs a *conjunctive symmetry breaker*: a formula that is true for one interpretation in every *orbit* of symmetric interpretations. This formula is conjunctively appended to the original formula, and then given to the solver. This method is sound because symmetries preserve models, so that either all points in an orbit are a model or none, whence it is sufficient for a solver to check (at least) one point per orbit. Most of these symmetry breaking formulas are based on the *lex-leader encoding* [8, 17, 20], which selects the smallest interpretation in every orbit based on a lexicographic ordering of the search space. In order to find symmetries of a formula, one typically encodes the formula into an appropriate graph structure and uses a graph theory package such as Saucy [10] to find its automorphisms. A schematic overview of the symmetry breaking process is shown in Figure 1.

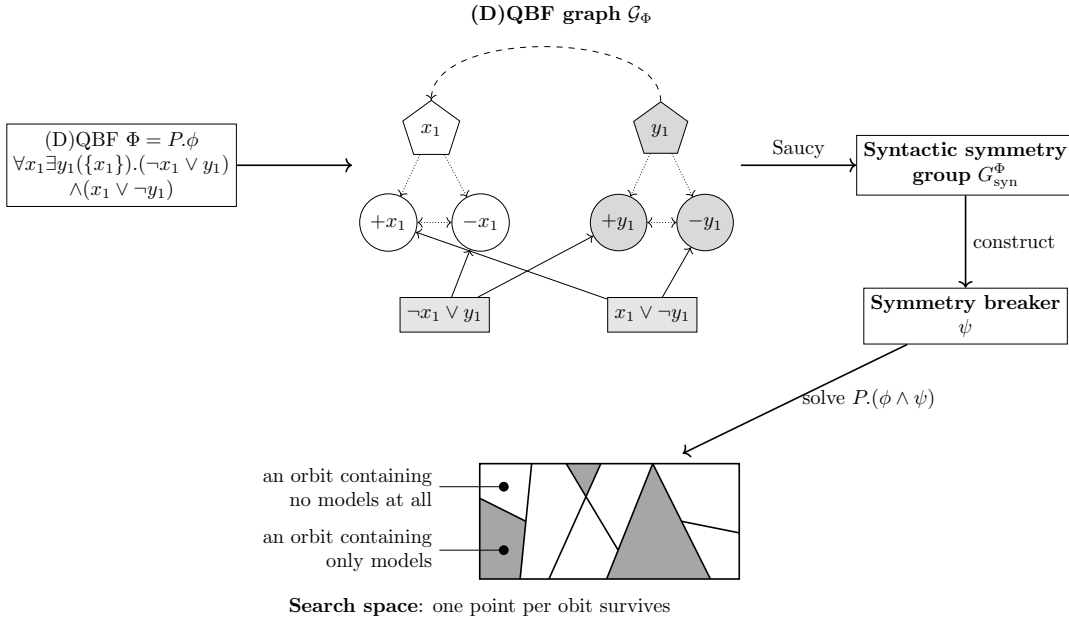


Figure 1: A schematic overview of symmetry breaking for (D)QBF.

It turns out that extending this framework to DQBF is non-trivial, due to the possible non-linear dependency structure of DQBFs. In the next section we present an example that shows that the symmetry breaker constructed in [17] does not lift to DQBFs in a straightforward way.

4 An Example

A DQBF is a generalization of a QBF where every existential quantifier is equipped with a *dependency set*, which determines the universal variables it may depend on. Due to this fine-grained control over the dependency structure, we only need one universal quantifier block followed by one existential quantifier block. Consider the DQBF

$$\Phi = P.\phi = \forall x_1, x_2, x_3 \exists y_1(\{x_2\}), y_2(\{x_3\}). (x_1 \leftrightarrow y_1) \vee (x_2 \leftrightarrow y_2) \vee (x_1 \leftrightarrow x_3).$$

The dependency sets are given by $D_1 = \{x_2\}$ and $D_2 = \{x_3\}$, which are not linearly ordered (i.e. $D_1 \not\subseteq D_2$ and $D_2 \not\subseteq D_1$). A model for a DQBF can be visualized as a tree which selects the value of the existential variables for every configuration (path) of universal variables, in such a way that on every branch the *matrix* ϕ is true. The dependency structure can be visualized by dashed arrows in such a tree: two nodes in the layer y_i are tied together if their configurations of the universal variables agree on the dependency set D_i . This is in fact an equivalence relation, and we call such clusters of points *dependency clusters*. Every model has to assign equal values to all points in a dependency cluster. A model s of Φ is visualized in Figure 2.

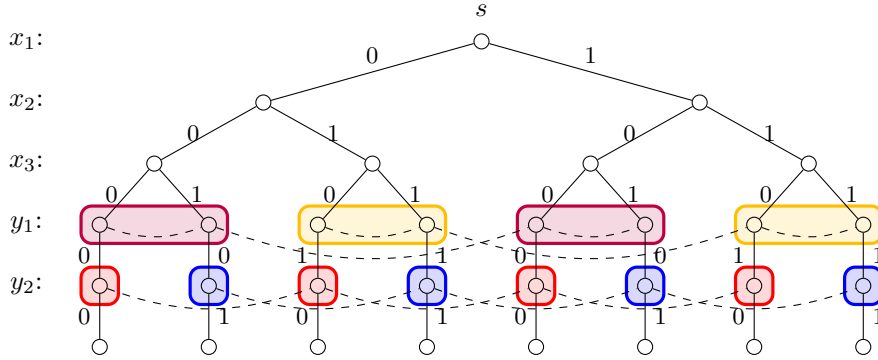


Figure 2: Model s of Φ , with coloured dependency clusters.

Formally, we use *Skolem functions* $s_j : \{0, 1\}^{|D_j|} \rightarrow \{0, 1\}$ ($j = 1, 2$) to specify the value of the existential variable y_j given the values of the universal variables $D_j \subseteq X$ that it depends on – in such a way that the formula becomes true under all assignments of the universal variables. Every Skolem function s_j can be represented by a Boolean formula with variables in D_j . A model s can then be represented as a tuple of Skolem functions, in Figure 2 we have $s = (s_1, s_2)$.

A symmetry is a transformation of an object that leaves a certain property *invariant*. It was already observed in SAT [20], CSP [9], and QBF [17] that there are two types of symmetries in this context: *syntactic symmetries* and *semantic symmetries*; for DQBF we use the same distinction. A syntactic symmetry, on the one hand, acts on the formula itself – a syntactic object. It is a transformation of the formula that preserves its syntactic structure in an appropriate sense. In our example a syntactic symmetry g of Φ is given by $g(x_2) = \neg x_2$ and $g(y_2) = \neg y_2$, while g is the identity on other variables. A semantic symmetry, on the other hand, acts on models of the formula – a semantic object. It transforms models of Φ into models of Φ . Moreover, these two notions of symmetry interact with each other: every syntactic symmetry induces a canonical semantic symmetry, though not vice versa.

In the example we get an induced semantic symmetry which swaps both components in s , $g(s) = (\neg x_2, \neg x_3)$, which is indeed again a model of Φ .

The adaptation of the common lex-leader encoding to DQBF for Φ is given by:

$$\begin{aligned} \psi := & (x_2 \leftrightarrow g(x_2)) \rightarrow (y_1 \rightarrow g(y_1)) \wedge \\ & (x_3 \leftrightarrow g(x_3) \wedge y_1 \leftrightarrow g(y_1)) \rightarrow (y_2 \rightarrow g(y_2)), \end{aligned}$$

which is equivalent to $\neg y_2$. Appending this to the original formula results in a false formula, so the usual symmetry breaking construction fails in this example. What is the problem? The proof of correctness of this encoding for QBF [17, Theorem 22] constructs a *mixed semantic symmetry* which applies the identity to the red part in Figure 3 and applies g to the blue part in Figure 3. However, such a mixed symmetry is not well-defined for the DQBF case since it is not acting uniformly on dependency clusters.

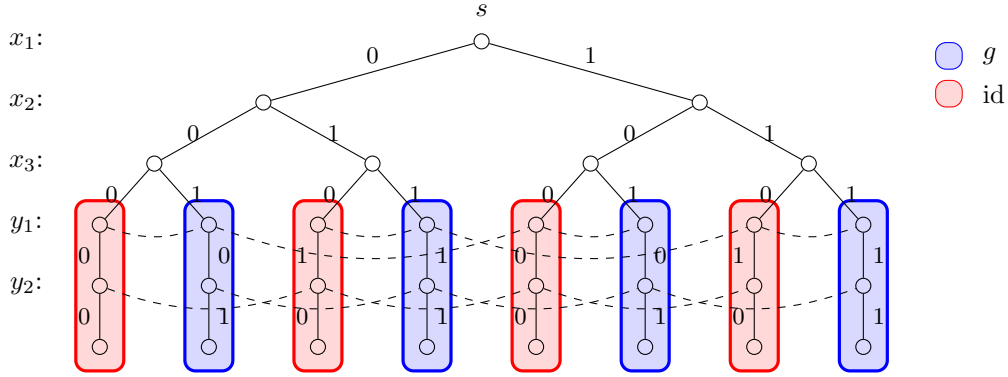


Figure 3: A mixed semantic symmetry acting on s .

5 Outline

In this talk we discuss symmetry breaking for DQBF and illustrate the difficulties arising from a non-linear dependency structure using the example. We sketch a solution, which is based on a more fine-grained ‘mixing theorem’ for semantic symmetries, containing a *gluing condition* with respect to the dependency structure. Moreover, the symmetry breaking theorem for DQBF contains an extra condition that mitigates nonlinearities in the dependency structure; it ensures that symmetries act trivially on these ‘non-linear parts’. This talk is based on work in progress, partly published in [15].

References

- [1] F.A. Aloul, K.A. Sakallah, and I.L. Markov. *Efficient Symmetry Breaking for Boolean Satisfiability*. IEEE Transactions on Computers 55(5), pp. 549–558 (2006)
- [2] G. Audemard, S. Jabbour, and L. Saïs. *Symmetry Breaking in Quantified Boolean Formulae*. In: Proceedings of International Joint Conference on Artificial Intelligence, pp. 2262–2267 (2007)
- [3] G. Audemard, S. Jabbour, and L. Saïs. *Efficient symmetry breaking predicates for Quantified Boolean Formulae*. In: Proceedings of Workshop on Symmetry and Constraint Satisfaction Problems, 7 pages (2007)

- [4] G. Audemard, B. Mazure, and L. Saïs. *Dealing with Symmetries in Quantified Boolean Formulas*. In: Proceedings of Theory and Applications of Satisfiability Testing. Online Proceedings (2004)
- [5] B. Bogaerts, S. Gocht, C. McCreesh, and J. Nordström. *Certified dominance and symmetry breaking for combinatorial optimisation*. Journal of Artificial Intelligence Research 77, pp. 1539–1589 (2023)
- [6] U. Bubeck. *Model-Based Transformations for Quantified Boolean Formulas*. PhD thesis, University of Paderborn (2009)
- [7] F.H. Chen, S.-C. Huang, Y.-C. Lu, and T. Tan. *Reducing NEXP-complete problems to DQBF*. In: Proceedings of FMCAD, pp. 199–204. TU Wien Academic Press (2022)
- [8] J.M. Crawford, M.L. Ginsberg, E.M. Luks, A. Roy. *Symmetry-Breaking Predicates for Search Problems*. In: Proceedings of the 5th International Conference on Principles of Knowledge Representation and Reasoning, pp. 148–159. Morgan Kaufmann (1996)
- [9] D. Cohen, P. Jeavons, C. Jefferson, K.E. Petrie, and B.M. Smith. *Constraint Symmetry and Solution Symmetry*. In: Proceedings of the National Conference on Artificial Intelligence, pp. 1589–1592. AAAI Press (2006)
- [10] T. Darga, M.H. Liffiton, K.A. Sakallah, and I.L. Markov. *Exploiting structure in symmetry detection for CNF*. In: Proceedings of the 41th Design Automation Conference (DAC), pp. 530–534. ACM (2004)
- [11] J. Devriendt, B. Bogaerts, M. Bruynooghe, and M. Denecker. *Improved static symmetry breaking for SAT*. In: Proceedings of Theory and Applications of Satisfiability Testing, pp. 104–122. Springer (2016)
- [12] J. Devriendt, B. Bogaerts, B. De Cat, M. Denecker, C. Mears. *Symmetry Propagation: Improved Dynamic Symmetry Breaking in SAT*. In: IEEE 24th International Conference on Tools with Artificial Intelligence, pp. 49–56. IEEE (2012)
- [13] L.H. Fung, C. Cheng, J.H.R. Jiang, F. Slivovsky, and T. Tan. *Model Counting for Dependency Quantified Boolean Formulas*. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, No. 17, pp. 14234–14242 (2026)
- [14] I.P. Gent, K.E. Petrie, and J. Puget. *Symmetry in Constraint Programming*. In: Handbook of Constraint Programming, pp. 329–376. Elsevier (2006)
- [15] C. Hofstadler, M. Kauers, and M. Seidl. *Symmetries of Dependency Quantified Boolean Formulas*. arXiv preprint: arXiv:2410.15848 (2024)
- [16] M. Kauers and M. Seidl. *Short proofs for some symmetric quantified Boolean formulas*. Information Processing Letters 140, pp. 4–7 (2018)
- [17] M. Kauers and M. Seidl. *Symmetries of Quantified Boolean Formulas*. In: International Conference on Theory and Applications of Satisfiability Testing (SAT), pp. 199–216. Springer (2018)
- [18] B. Krishnamurthy. *Short Proofs for Tricky Formulas*. Acta informatica 22, pp. 253–275 (1985)
- [19] G. Peterson, J. Reif, and S. Azhar. *Lower Bounds for Multiplayer Non-Cooperative Games of Incomplete Information*. Computers and Mathematics with Applications 41(7–8), pp. 957–992 (2001)

- [20] K.A. Sakallah. *Symmetry and Satisfiability*. In: Handbook of Satisfiability, 2nd edition, pp. 289–338. IOS Press (2021)
- [21] C. Scholl and R. Wimmer. *Dependency Quantified Boolean Formulas: An Overview of Solution Methods and Applications*. In: International Conference on Theory and Applications of Satisfiability Testing, pp. 3–16. Springer (2018)
- [22] R.K. Tchinda and C.T. Djamegni. *On certifying the UNSAT result of dynamic symmetry-handling-based SAT solvers*. Constraints 25(3), pp. 251–279 (2020)
- [23] A. Urquhart. *The symmetry rule in propositional logic*. Discrete Applied Mathematics 96, pp. 177–193 (1999)